#1 Method `dbscan` takes a list of points, the radius of the dense area defining core points, and the minimum number of points needed in the dense area for a point to be a core point. This function returns an array of the cluster indices associated with the points (or, equivalently, a dictionary associating points to cluster indices).
#2 Initialize the current cluster's index to `0`. We will use the special value `0` to mark a point as unprocessed, while the other special value `-1` will be used to flag outliers. Valid cluster indices start from `1`.
#3 Initialize list with the cluster index for each point. At first, each point is marked as unprocessed.
#4 Create a k-d tree to speed range queries. It initializes it with the full dataset.
#5 Cycle through each point in the dataset.
#6 If `p`'s cluster index is not `0` anymore, it means that the point has already been processed, and so it can be skipped here.
#7 Initializes the set of points that will have to be processed while constructing the current cluster. Initially, `p` is the only point in the dataset.
#8 Mark `p` as processed, by initially flagging it as an outlier.
#9 We are creating a new cluster, so we can increment the index of current cluster. This implementation doesn't worry about having all cluster's indices as consecutive integers: in other words, whenever we find out that `p` is an outlier, we skip current index. This can be easily fixed, for instance by using a Boolean flag.
#10 Cycle through each point `q` in the list of points to process.